

Reviewer Pack — Minimal Local Coherence Index in Quandles and DPLL Proof Len...

Entry 6d01ebad94e6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 15:59:29 UTC

Minimal Local Coherence Index in Quandles and DPLL Proof Length Correlation

Entry ID: 6d01ebad94e6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 15:59:29 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Quandles) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Length)

Statement:

The minimal local coherence index of a quandle associated with a CNF is linearly correlated with the DPLL proof length of the CNF, such that $\text{LocalCoherenceIndex}(Q) = \Theta(\text{DPLLProofLength}(\text{CNF}))$.

Rationale (proposer's reasoning):

Quandles provide a categorical framework for studying algebraic structures that exhibit certain group-like properties. The minimal local coherence index captures the complexity of these structures. A higher correlation between this index and DPLL proof length suggests a deeper connection between quandle theory and computational complexity, potentially leading to new insights into proof complexity.

Taxonomy category: Quandle_Theoretical (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f5f50344c6f25870

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the correlations between minimal local coherence index and DPLL proof length are within a threshold of ± 3 , across at least 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local coherence index" AND quandle AND DPLL proof length" - "CNF associated quandle" AND linear correlation AND DPLL proof length" - "Category Theory Quandles" AND "Boolean Satisfiability DPLL proof length"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]

    # Back-substitute
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def dpll(cnf):
    if not cnf:
        return True
    literals = set()
    for clause in cnf:
```

```

        literals.update(abs(x) for x in clause)
        literal = random.choice(list(literals))
        new_cnf = [[x for x in clause if x != literal and x != -literal] for clause in cnf]
        return dpll(new_cnf) or dpll([-x] for x in literals)

def construct_quandle(cnf):
    n = len(cnf)
    quandle = {}
    for i in range(1, n+1):
        quandle[i] = {j: (i + j - 1) % n + 1 for j in range(1, n+1)}
    return quandle

def minimal_local_coherence_index(quandle):
    n = len(quandle)
    index = 0
    for i in range(1, n+1):
        for j in range(1, n+1):
            if quandle[i][j] != quandle[j][i]:
                index += 1
    return index / (n * (n - 1))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    trials = 30
    instances_tested = 0
    n_max = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for _ in range(trials):
        n = random.choice([5, 10, 15, 20, 30, 40])
        cnf = []
        for _ in range(n):
            clause = [random.randint(1, n) * (-1 if random.random() < 0.5 else 1)
                       for _ in range(random.randint(1, n))]
            cnf.append(clause)

        quandle = construct_quandle(cnf)
        local_coherence_index = minimal_local_coherence_index(quandle)
        proof_length = dpll(cnf) * len(cnf)

        if proof_length == 0:
            conjecture_holds = False
            counterexample = "DPLL proof length is zero"
            break

        metric_value = abs(local_coherence_index - proof_length)
        total_metric_value += metric_value
        instances_tested += n
        n_max = max(n_max, n)

    if not conjecture_holds:
        return {
            "metric_name": "LocalCoherenceIndex-DPLLProofLengthCorrelation",
            "metric_value": 0.0,

```

```

        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": counterexample
    }

    mean_metric_value = total_metric_value / instances_tested
    threshold = 3 * (1 / math.sqrt(instances_tested))
    if any(metric_value > mean_metric_value + threshold or metric_value < mean_metric_value - threshold):
        conjecture_holds = False

    return {
        "metric_name": "LocalCoherenceIndex-DPLLProofLengthCorrelation",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] and r["counterexample"] for r in results):
        counterexamples = [r["counterexample"] for r in results if not r["conjecture_holds"]]
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{' '.join(counterexamples)}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE no_counterexamples_found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ble2ef3b.py", line 127, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ble2ef3b.py", line 87, in run_trial
    proof_length = dpll(cnf) * len(cnf)
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ble2ef3b.py", line 50, in dpll
    return dpll(new_cnf) or dpll([[-x] for x in literals])

```

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ble2ef3b.py", line 50, in dpll
    return dpll(new_cnf) or dpll([[x] for x in literals])
^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ble2ef3b.py", line 50, in dpll
    return dpll(new_cnf) or dpll([[x] for x in literals])
^^^^^^^^^^^^^^^^
[Previous line repeated 17 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ble2ef3b.py", line 48, in dpll
    literal = random.choice(list(literals))
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')
IndexError: Cannot choose from an empty sequence

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required number of trials to evaluate the conjecture's support condition. | next: Re-run the test with proper error handling and ensure that it completes all necessary trials to validate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13815
2	preregistration	ollama_remote	glm4:latest	0	0	9140
3	novelty	ollama_remote	glm4:latest	0	0	12682
4	novelty	ollama_remote	glm4:latest	0	0	9341
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44924
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12552
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12491
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15833
9	judge	ollama_remote	glm4:latest	0	0	15250

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146028 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/6d01ebad94e6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6d01ebad94e6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6d01ebad94e6.tar.gz` (if generated)